DevOps Sprint 1

Deployment Architecture & Branching Strategy

Name: Sandil Helitha Perera

IT Number: IT23657700

March 2, 2026

Contents

1 Deployment Architecture Components **2**
- 1.1 Backend: Azure App Service (F1 Tier) . . . 2
- 1.2 Frontend: Azure Static Web Apps (Free Tier) . . . 2
- 1.3 Database: Azure SQL (Student/Free Tier) . . . 3

2 Deployment Architecture **3**

3 Best Practices for Deployment **4**

4 Cost Summary **4**

5 Known Limitations **4**

6 Branch Structure & Hierarchy **6**
- 6.1 Main Branch . . . 6
- 6.2 Staging Branch . . . 6
- 6.3 Develop Branch . . . 7
- 6.4 Feature Branches . . . 7
- 6.5 Testing Branch . . . 7
- 6.6 Pre-staging Branch . . . 8

7 Workflow Guidelines **8**
- 7.1 Pull Request Process . . . 8
- 7.2 Merge Strategies . . . 8
- 7.3 Commit Message Standards . . . 9

8 Branch Naming Conventions **9**
- 8.1 Feature Branches . . . 9
- 8.2 Bugfix Branches . . . 9
- 8.3 Hotfix Branches . . . 10
- 8.4 Release Branches . . . 10

9 Continuous Integration & Deployment **10**
- 9.1 Automated Testing . . . 10
- 9.2 Deployment Pipeline . . . 11

10 Best Practices & Guidelines for Branching **11**
- 10.1 General Best Practices . . . 11
- 10.2 Security Considerations . . . 12

11 Github Actions **12**

OVERVIEW

This document outlines the deployment architecture for hosting .NET 10 web applications using Azure's free tier services and the development branching strategy for GitHub repositories.

1 DEPLOYMENT ARCHITECTURE COMPONENTS

1.1 Backend: Azure App Service (F1 Tier)

Azure App Service F1 (Free) Tier provides a fully managed hosting environment for your .NET 10 web API with the following specifications:

Specification	Details
Cost	$0 (Free)
Memory	1 GB RAM
Storage	1 GB
CPU Limit	60 CPU minutes per day
Platform	Linux (Suitable for .NET 10)

Key Features

- The 60 CPU minutes per day limit is ideal for development and testing workloads.
- Linux stack recommended for better performance and compatibility with modern .NET features.
- Supports continuous deployment from GitHub, Azure DevOps, and other repositories.

1.2 Frontend: Azure Static Web Apps (Free Tier)

Azure Static Web Apps is specifically designed for modern frontend frameworks including Angular and React js. It provides enterprise level grade hosting with an exceptional developer experience.

Specification	Details
Cost	$0 (Free)
Bandwidth	100 GB per month
SSL Certificate	Free (Auto-provisioned)
CI/CD	Native GitHub Actions integration
Scaling	Automatic global distribution

Key Features

- Automatically scales to handle traffic spikes with global CDN distribution.
- Free SSL certificates with automatic renewal.
- Seamless GitHub Actions integration for automated deployments.
- Custom domain support with automatic HTTPS.

1.3 Database: Azure SQL (Student/Free Tier)

Azure SQL Database provides enterprise level grade relational database capabilities with generous free tier allocations for students and developers.

Specification	Details
Cost	$0 (Student Benefit)
Compute	100,000 vCore seconds (Serverless)
Storage	32 GB
Backup	Automated with 7-day retention

Key Features

- Serverless compute model.
- Automatic backups with point-in-time restore capability.
- Built-in high availability and security features.
- 32 GB storage is sufficient for most development.

2 DEPLOYMENT ARCHITECTURE

The following diagram illustrates how these components work together:

- **Frontend (React)** → Azure Static Web Apps → Global CDN
- **Backend (.NET 10 API)** → Azure App Service (F1) → Linux Container
- **Database** → Azure SQL (Serverless) → Automated Backups
- Frontend communicates with Backend via HTTPS REST API
- Backend connects to Database using secure connection strings

3 BEST PRACTICES FOR DEPLOYMENT

For Backend (App Service)

- Prioritize the Linux runtime stack to improve performance and efficiency.
- To avoid interruptions, monitor daily CPU limits.
- Implement efficient caching strategies to reduce CPU usage.
- Use Application Insights (free tier) for monitoring and diagnostics.

For Frontend (Static Web Apps)

- Configure custom domains early in development.
- Use environment variables for API endpoints to support multiple environments.
- Enable preview environments for pull requests to test changes before merging.

For Database (Azure SQL)

- Configure serverless auto-pause to minimize compute usage when idle.
- Store connection strings securely in App Service configuration.
- Implement database indexes for frequently queried columns.

4 COST SUMMARY

Service	Monthly Cost
Azure App Service (F1)	$0.00
Azure Static Web Apps (Free)	$0.00
Azure SQL (Student)	$0.00
Total Monthly Cost	**$0.00**

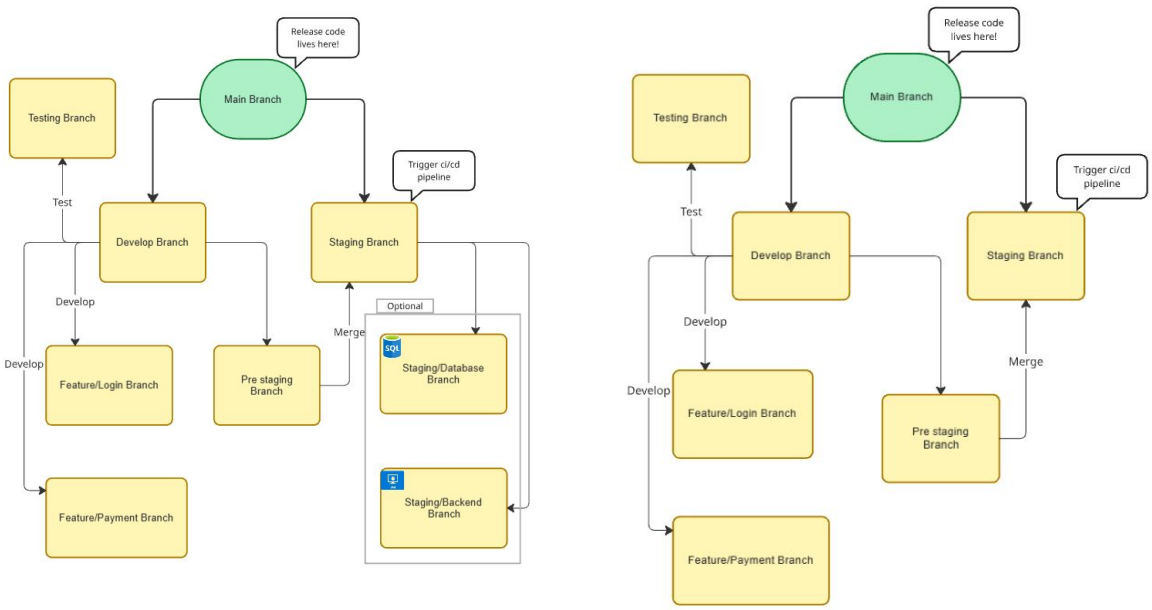
5 KNOWN LIMITATIONS

While this architecture is excellent for development and small scale projects, be aware of these limitations:

- **60 CPU minutes per day** on App Service - monitor usage for high-traffic scenarios.
- **No SLA guarantees** on F1 tier - not suitable for production workloads.

- **Cold start delays** - first request after idle period may be slower.
- **100,000 vCore seconds** monthly limit on Azure SQL - monitor for data-intensive operations.
- **100 GB bandwidth** limit on Static Web Apps free tier - sufficient for most small scale projects.

6 BRANCH STRUCTURE & HIERARCHY



(a) Backend Branching (b) Frontend Branching

Figure 1: Overview of Development Branching Strategies

6.1 Main Branch

Purpose: The main branch represents the production-ready state of the codebase. This includes main release of the project. This branch is always stable and deployable.

Protection Rules:

- Direct commits are prohibited.
- All CI/CD checks must pass.
- Only staging branch can merge into main.

6.2 Staging Branch

Purpose: Pre-production environment for final testing and validation before production deployment. Automatically triggers CI/CD pipelines for deployment to staging environment. Cloud is connected to this branch for the deployment.

Key Characteristics:

- Receives merges from pre-staging branch (frontend) or direct merges from develop (backend when pre-staging not used).
- Automated deployment to staging environment.

- Triggers automated testing suites.
- Requires successful validation before main branch merge.

6.3 Develop Branch

Purpose: Primary integration branch where feature branches are merged. Serves as the main development line.

Workflow:

- All feature branches branch from develop.
- Feature branches merge back to develop upon completion.
- Regular integration testing performed.
- Merges to pre-staging or staging for deployment.

6.4 Feature Branches

Purpose: Isolated development environments for individual features or user stories. This branch naming is created using the jira user story.

Examples:

- `CSP_12_Login Branch`: User authentication feature
- `CSP_15_Payment Branch`: Payment processing functionality

Lifecycle:

- Created from develop branch.
- Developed and tested independently.
- Merged back to develop via pull request.
- Deleted after successful merge.

6.5 Testing Branch

Purpose: Dedicated branch for quality assurance testing and bug verification before staging deployment.

Usage:

- Receives merges from develop branch.
- QA team performs comprehensive testing.
- Bugs are fixed in feature branches and re-merged.

- Only validated code proceeds to staging.

6.6 Pre-staging Branch

Purpose: Intermediate branch between development and staging, providing an additional validation layer before staging deployment. Development code is merged to this branch and adjusted for the staging branch.

When to Use:

- Frontend repository (standard workflow).
- Backend repository when additional validation needed.
- Large releases requiring staged rollout.

7 WORKFLOW GUIDELINES

7.1 Pull Request Process

Creating Pull Requests:

- Provide clear, descriptive title summarizing changes.
- Include detailed description with context and rationale.
- Reference related tickets/issues (e.g., JIRA-123, #456).
- List testing performed and results.
- Add screenshots or videos for UI changes.

Review Requirements:

- Minimum 1 approval for feature branches to develop.
- Minimum 2 approvals for staging to main.
- All CI/CD checks must pass (build, tests, linting).
- No merge conflicts allowed.

7.2 Merge Strategies

- **Squash and Merge:** Recommended for `feature` → `develop` (clean history).
- **Merge Commit:** For `develop` → `staging`, `staging` → `main` (preserve branch history).
- **Rebase:** Use cautiously, only for private feature branches not pushed to remote.

7.3 Commit Message Standards

Format:

```
<type>(<scope>): <subject>
```

Types:

- **feat:** New feature
- **fix:** Bug fix
- **docs:** Documentation changes
- **style:** Code style/formatting (no logic change)
- **refactor:** Code refactoring
- **test:** Adding or updating tests
- **chore:** Build process or auxiliary tool changes

Examples:

```
feat(auth): add password reset functionality
fix(api): resolve null pointer in user service
docs(readme): update installation instructions
```

8 BRANCH NAMING CONVENTIONS

8.1 Feature Branches

Format: `CSP_{User story no}_{User story name}`

Examples:

- `CSP_10_user-authentication`
- `CSP_15_payment-integration`
- `CSP_18_dashboard-widgets`

8.2 Bugfix Branches

Format: `bugfix/<description>` or `fix/<description>`

Examples:

- `bugfix/login-redirect-error`
- `fix/memory-leak-user-service`

8.3 Hotfix Branches

Format: `hotfix/<description>`

Usage: Critical production fixes that cannot wait for next release.

Examples:

- `hotfix/critical-security-patch`
- `hotfix/payment-processing-failure`

Workflow:

- Branch from main (production).
- Fix the critical issue.
- Merge to main and tag with version.
- Merge back to develop and staging to keep branches synchronized.

8.4 Release Branches

Format: `release/<version>`

Examples:

- `release/v1.2.0`
- `release/v2.0.0-beta`

9 CONTINUOUS INTEGRATION & DEPLOYMENT

9.1 Automated Testing

Pull Request Checks:

- Unit tests (must achieve >80% code coverage).
- Integration tests.
- Linting and code style checks.
- Build verification.
- Security vulnerability scanning.

Staging Deployment Checks:

- End-to-end tests.

- Performance tests.
- Smoke tests post-deployment.

9.2 Deployment Pipeline

Frontend Repository

- Merge to staging branch triggers CI/CD.
- Build process: `npm install`, `webpack build`.
- Run automated test suite.
- Deploy to staging environment (CDN, static hosting).
- Run smoke tests, notify team of deployment status.

Backend Repository

- Merge to staging branch triggers CI/CD.
- Build Docker container or package application.
- Run database migrations (if staging/database branch used).
- Deploy to staging servers (rolling deployment).
- Health checks and smoke tests.
- Monitoring and alerting active.

10 BEST PRACTICES & GUIDELINES FOR BRANCHING

10.1 General Best Practices

- Keep feature branches short-lived.
- Regularly sync feature branches with develop to minimize conflicts.
- Write meaningful commit messages that explain the 'why'.
- Delete feature branches after successful merge.
- Never force push to shared branches (main, staging, develop).
- Use draft pull requests for early feedback.
- Tag production releases with semantic versioning (v1.2.3).

10.2 Security Considerations

- Never commit secrets, API keys, or credentials.
- Use environment variables for configuration.
- Enable branch protection rules on main and staging.
- Require signed commits for production branches.
- Regular dependency audits and updates.

11 GITHUB ACTIONS

- **Trigger Mechanism:** Ensures that new feature contributions, such as the Internship Recruitment or academic innovation modules, do not introduce breaking changes to the main development line.
- **Checkout Code:** Uses the `actions/checkout` action to pull the latest repository source code into the virtual runner environment.
- **Setup .NET 10 SDK:** Configures the environment with the specific **.NET 10 SDK** required for the project, ensuring consistency between local development and cloud build.
- **Restore Dependencies:** Executes `dotnet restore` to fetch all necessary **NuGet packages**, resolving external dependencies before the compilation phase.
- **Build (Release Mode):** Compiles the project using the **Release configuration**. This produces an optimized, production-ready binary and catches strict compile-time errors that might be missed in debug builds.
- **Automated Testing:** A modular step intended for executing **unit and integration tests** (e.g., xUnit). It ensures that core logic, such as **Role-Based Access Control (RBAC)** and application status transitions, remains functional.

CONCLUSION

This Azure free tier architecture provides a complete, production-ready stack for deploying .NET 10 web applications at zero cost. The combination of App Service (F1), Static Web Apps, and Azure SQL delivers enterprise-grade features with generous limits for the Internova project. As your projects grow, you can seamlessly upgrade to paid tiers without changing your application architecture. Comprehensive planned github branching strategy ensures smooth and seamless development process while maintaining the correct code.